# BroStatus v1.0.0 by DiamondSash

**El indicador de estado para Claude Code:** Un diamante te muestra con un color si Claude está trabajando, te está esperando o ya terminó. Al lado ves el contexto, la caché del prompt, las herramientas usadas y los archivos cambiados. Encima del campo de escritura hay botones para responder.

Fecha 04.11.2026 · Versión 1.0.0 · probado con Claude Code 2.1.310 · Windows | Mac | Linux · sin servidores propios

**Dónde funciona:** solo en Claude Code: en el terminal (`claude`) en Windows, Mac y Linux, en la aplicación de escritorio de Claude en «Code» y en editores como VS Code con la extensión de Claude Code. **Probado** (con mis propias pruebas): Windows 10 con la aplicación de escritorio de Claude en uso real; la edición sin las funciones automáticas aún no la he visto en la interfaz real; comparte el código con la edición completa, que probé el 30/10/2026 en el terminal (PowerShell) y en la aplicación de escritorio (diamante, panel), VS Code en Windows (el 29/10/2026: panel, estado, moneda) y el instalador en un segundo PC con Windows. Mac: lo ha probado una persona en el terminal, la aplicación de escritorio en Mac aún no está probada. Linux: el instalador se incluye, aún no se ha probado en la práctica, los comentarios son bienvenidos. No funciona en el chat normal de Claude ni en otras herramientas de IA (por ejemplo Cursor, Codex, Gemini).

| 5 colores | 3 formas | 0 tokens |
|---|---|---|
| tiene el diamante: de rojo (crítico) a verde (listo), siempre se ve el más importante | para avisarte: título de la ventana, aviso en el terminal, Telegram al móvil | cuesta el propio indicador (un token es la unidad con la que Claude cuenta y cobra el texto) |

**No incluido en esta edición:** mantener la caché caliente, Auto-Compact, Fin de jornada y Cambio de modelo. Estas funciones envían solicitudes por su cuenta a través de tu cuenta. Si eso encaja con las condiciones de los planes de Anthropic, se ha preguntado a Anthropic y aún no hay respuesta; llegarán solo cuando se aclare.

## Lo que ganas

Lo que BroStatus hace por ti.

### Cuándo te toca

El diamante se vuelve gris en cuanto Claude espera por ti. En ejecuciones largas te avisa el ordenador o el móvil.

El diamante · Señales

### Caché y costes

Ves la cuenta atrás de la caché. Si está fría, lo ves y sabes por qué el siguiente mensaje cuesta más; si quieres evitarlo, envías tú mismo un mensaje a tiempo.

Línea de caché

### Ver el contexto

El semáforo avisa a tiempo antes de que el contexto se llene. Cuando toca, resumes tú con `/compact`.

Semáforo de contexto

### Responder preguntas

Un botón por punto, escribes la respuesta y lo envías todo junto. Claude recibe cada respuesta con su pregunta.

Botones de respuesta

### Límites y coste

Ves cuánto llevas gastado de tus límites de 5 horas y semanal y el valor de API estimado de la sesión, también en la moneda que elijas.

El panel /bro

### Qué hace Claude

BroStatus muestra en directo qué herramientas se usan y qué archivos se cambian, son nuevos o solo se leen.

Activo y archivos

### Ejemplo: caché

Estás trabajando en algo, te distraes y te olvidas del chat. La caché, la memoria temporal de la parte de la conversación que Claude ya ha leído, caduca tras una pausa. BroStatus te muestra la cuenta atrás. Si la caché está fría, lo ves y sabes por qué tu siguiente mensaje cuesta más: hay que escribir de nuevo todo el historial. Si ves que está a punto de caducar y quieres seguir, envías tú mismo un mensaje a tiempo. Tu conversación no se pierde en ningún caso.

### Ejemplo: compact

Programas sin parar y no ves el contexto. Cuanto más se llena, más bajan la precisión y la memoria (a menudo llamado «context rot»). La barra de contexto te muestra cuánto está lleno. El semáforo avisa y tú resumes con `/compact` cuando te convenga; si no, Claude Code compactará pronto por sí mismo cuando el contexto se acerque al límite.

### Ejemplo: sonidos

Estás haciendo otra cosa mientras Claude trabaja. Cuando Claude pide una aprobación o muestra una elección, BroStatus reproduce un sonido de aviso en la app de escritorio y puedes volver y responder. Cuando Claude termina tras un trabajo largo, suena un sonido suave. Ambos se ajustan en «Más» o se desactivan. En el terminal suena en su lugar la campana del terminal.

**En local y gratis:** El indicador funciona sin servidores propios, no envía nada al autor y no gasta tokens. BroStatus no envía solicitudes a Claude por su cuenta a través de tu cuenta. Solo esto sale de tu ordenador: Telegram (solo si lo configuras tú, con tu propio bot) y, en Windows, la descarga de Python con winget (el instalador acepta por ti las condiciones de winget y de Python). La carpeta `state/` guarda archivos de estado locales con rutas de archivos y el comienzo de comandos.

## Contenido

Todas las imágenes de la línea de estado vienen directamente de BroStatus.


01 De un vistazo
02 El diamante
03 Línea de estado
04 Semáforo de contexto
05 Línea de caché
06 Activo y archivos
07 Botones de respuesta
08 El panel /bro
09 Señales de listo y de espera
10 Instalación
11 Comprobar que todo funciona
12 Ajustes
13 Actualizaciones y solución de problemas


## De un vistazo

Así se ve BroStatus abajo en Claude Code mientras Claude trabaja.




Salida real de la línea de estado con datos de ejemplo: Claude lleva trabajando 1 minuto y 12 segundos y ahora está ejecutando pruebas.

| | |
|---|---|
| **DIAMANTE** | A la izquierda está el emblema. Su color te dice en un segundo si te necesitan. Más información |
| **ESTADO** | Qué está haciendo Claude, desde cuándo, qué herramienta está en marcha, además del modelo y el nivel de razonamiento. Más información |
| **CONTEXTO** | Cuánto se ha llenado la memoria de la conversación, con semáforo y un consejo sobre cuándo conviene limpiar. Más información |
| **CACHÉ** | Cuánto tiempo sigue caliente la caché del prompt, qué tal acierta y cuánto llevas gastado de tus límites. Más información |
| **ACTIVO** | Qué herramientas, skills, servidores MCP, direcciones web y subagentes se han usado en esta sesión. Más información |
| **ARCHIVOS** | Qué archivos ha cambiado, creado o solo leído Claude. Más información |

## El diamante

Siempre un solo color. Si se cumplen varias cosas, gana la más importante.



**Rojo · crítico**

Error de API, contexto desde el 85 % o la caché caduca en los próximos 2 minutos (caché de 1 h) o 30 segundos (caché de 5 min).



**Gris · te toca a ti**

Claude espera tu aprobación o tu respuesta. Plateado claro, para que se note.



**Naranja · actúa pronto**

Contexto desde el 50 % o la caché caduca pronto (caché de 1 h: menos de 10 minutos, caché de 5 min: menos de 1 minuto).



**Azul · trabajando**

Claude está en plena respuesta. No tienes que hacer nada.



**Verde · listo**

La respuesta ya está, todo en orden. También en reposo, «preparado».

Orden de prioridad: rojo › gris › naranja › azul › verde

**Ejemplo:** Claude trabaja, pero el contexto está al 58 %. Entonces el diamante es naranja en vez de azul, porque «actúa pronto» es más importante que «trabajando». Los límites del contexto los cambias con `ctx_stages` en los Ajustes.

## Línea de estado

La primera línea dice qué está haciendo Claude ahora.

**PREPARADO**

Todavía no ha pasado nada en esta sesión. A la derecha están el modelo y el nivel de razonamiento, aquí Opus con «high».

```
○ preparado · opus/high
```

**TRABAJANDO**

Claude lleva trabajando 1 minuto y 12 segundos. Detrás de la flecha está la herramienta en marcha con su tiempo. Si hay varias a la vez, detrás pone «+2 en paralelo».

```
⚙ trabajando 1m 12s → Bash: npm test 8s · opus/high
```

**PENSANDO**

Claude trabaja, pero ahora no usa ninguna herramienta.

```
⚙ trabajando 5s → pensando… · opus/high
```

**ESPERÁNDOTE**

Claude te necesita, casi siempre para una aprobación. Detrás pone qué está esperando Claude. El diamante se pone gris.

```
‖ esperándote – Bash: git push · opus/high
```

**LISTO**

La respuesta ya está: terminó hace 12 segundos, tras 3 minutos y 20 segundos de trabajo.

```
✓ listo hace 12s · duró 3m 20s · opus/high
```

**EN SEGUNDO PLANO**

Claude ya respondió, pero todavía se ejecuta un comando o un agente auxiliar, aquí dos. En lugar de «listo» aparece «trabajando en segundo plano» con la primera tarea, y el diamante sigue azul. En el panel hay debajo una línea por tarea. Solo cuando todo ha terminado aparece «listo», con sonido.

```
◔ trabajando en segundo plano · 2 → npm run build +1 · hace 40s · opus/high
```

**CANCELADO**

La respuesta se cortó por un error de la interfaz. Casi siempre ayuda volver a enviar el mensaje.

```
× cancelado (error de API) hace 3s · opus/high
```

## Semáforo de contexto

Cuánto se ha llenado la memoria de la conversación, con un consejo de lo que toca hacer.

**MENOS DEL 50 %**

Cian: todo tranquilo. Detrás del porcentaje están los tokens usados y los posibles.

```
contexto▮▮▮▮▮▯▯▯▯▯▯▯▯▯▯ 30% 60k/200k
```

**DESDE EL 50 %**

Amarillo: **terminar la tarea**. Acaba la tarea en curso antes de empezar algo nuevo.

```
contexto▮▮▮▮▮▮▮▮▮▯▯▯▯▯▯▯ 58% 116k/200k ▸ terminar la tarea
```

**DESDE EL 70 %**

Naranja: **/compact o chat nuevo**. Ahora es un buen momento para limpiar.

```
contexto▮▮▮▮▮▮▮▮▮▮▮▮▯▯▯▯ 75% 150k/200k ▸ /compact o chat nuevo
```

**DESDE EL 85 %**

Rojo: **¡pronto compactar!** Claude Code compactará pronto por sí mismo la conversación; mejor hazlo tú antes con `/compact`.

```
contexto▮▮▮▮▮▮▮▮▮▮▮▮▮▮▯▯ 88% 176k/200k ▸ ¡pronto Auto-Compact!
```

**BUEN MOMENTO**

Si la caché ya está fría y Claude no está ocupado, un `/compact` cuesta apenas más que la reconstrucción que toca de todos modos. Entonces aparece el aviso verde.

```
contexto▮▮▮▮▮▮▮▮▮▮▮▮▯▯▯▯ 75% 150k/200k ▸ /compact o chat nuevo (caché fría → buen momento)
```

## Línea de caché

La caché del prompt ahorra dinero y cuota mientras está caliente.

**Tu ventaja:** Sabes cuánto puedes parar sin que tu siguiente mensaje salga caro.

Anthropic guarda durante un tiempo el historial de la conversación. Mientras esa caché está caliente, el historial solo se lee en tu siguiente mensaje, y eso cuesta más o menos una décima parte (menos en algunos modelos). Si ha caducado, hay que escribirlo todo de nuevo.

**CALIENTE**

La cuenta atrás muestra cuánto aguanta aún la caché, y al lado la duración ( `1h` o `5m` ).

```
caché  ◷ 54:58 1h  aciertos 99% última · 90% sesión  límite · S | 5h / 23%  · Sem | 7d / 71%  #tools_c…
```

**CADUCA PRONTO**

Amarillo cuando a la caché de 1 h le quedan menos de 10 minutos. Con la caché de 5 min aquí no hay amarillo, solo en el panel.

```
caché  ◷ 8:05 1h · caduca pronto  aciertos 99% última · 90% sesión  límite · S | 5h / 23%  · Sem | 7d /…
```

**CADUCA ENSEGUIDA**

Rojo en los últimos 2 minutos; con la caché de 5 min, en el último minuto. Si quieres seguir, envía tu mensaje ahora.

```
caché  ◷ 1:34 1h · caduca pronto  aciertos 99% última · 90% sesión  límite · S | 5h / 23%  · Sem | 7d /…
```

**FRÍA**

Caducada. «nuevo ~92k» significa: tu siguiente mensaje vuelve a escribir unos 92.000 tokens en la caché.

```
caché  ◷ frío 1h · nuevo ~92k  aciertos 99% última · 90% sesión  límite · S | 5h / 23%  · Sem | 7d / 71…
```

**CACHÉ DE 5 MIN**

Según tu cuenta y tus ajustes, la caché puede durar solo 5 minutos (a menudo con clave de API o crédito adicional).

```
caché  ◷ 3:31 5m  aciertos 99% última · 90% sesión  límite · S | 5h / 23%  · Sem | 7d / 71%  #tools_ch…
```

### Aciertos

- **última:** parte de la última petición que vino de la caché
- **sesión:** lo mismo para toda la sesión
- Alto es bueno. 99 % significa que casi todo se leyó barato

### Límite

- Gasto de tu suscripción: **ventana de 5 horas / semana**
- Blanco hasta el 64 %, amarillo desde el 65 %, rojo desde el 85 %

### #Motivos

- Por qué hubo que reescribir la caché la última vez
- `#tools_changed×2` : la lista de herramientas ha cambiado dos veces

## Activo y archivos

Qué se ha usado y tocado en esta sesión.

```
activo  read×6  edit×3  bash×2  grep×2  ◇ pdf  ⌁ github×2  ↗ api.github.com  ⬡ Explore
```

### Símbolos en la línea Activo

- **sin símbolo:** herramientas como read, edit, bash, con cantidad ( `read×6` )
- ◇ skills que Claude ha cargado
- ⌁ servidores MCP, es decir, servicios conectados
- ↗ direcciones web y búsqueda en la web
- ⬡ subagentes que han ayudado
- De cada tipo salen como máximo 5 entradas, el resto como `+3`

### Símbolos en los archivos

- \+ escrito (nuevo o sobrescrito)
- ✎ cambiado
- · solo leído
- Los cambiados van primero, y siempre los más recientes antes

```
archivos✎ statusline.py  + hilfe.py  ✎ README.md  · anleitung.md  · config.json
```

Vista compacta (la normal): una línea, lo que no cabe aparece como `+2` al final.

```
├─ docs/
│  └─ · anleitung.md
├─ src/
│  ├─ neu/
│  │  └─ + hilfe.py
│  ├─ · config.json
│  └─ ✎ statusline.py
└─ ✎ README.md
```

Con `"files": "tree"` aparece debajo un árbol de archivos de hasta 8 líneas.

## Botones de respuesta

Mod «bro-status»: responde a los puntos abiertos sin escribirlo todo.

Si la respuesta de Claude termina con una lista numerada o con viñetas que te pide algo, encima del campo de escritura aparece una banda con un botón por cada punto. Respondes los puntos uno a uno y «Send» envía todo junto como tu mensaje.

```
◆ Respuestas  1/2 respondidas  [ Send (1) ]  [ Hide ]
[ Aún faltan 4 imágenes ]   ✓ ya están en la carpeta
[ Esto todavía lo tienes que hacer tú ]
Sobre 2 está hecho                                  aceptar
```

Recreado a partir del código de la mod. En Claude Code la banda puede verse algo distinta según el terminal.

### Así se hace

1. Haz clic en el botón del punto
2. Escribe la respuesta en el campo «Sobre 1» y pulsa `Enter`. Salta al siguiente punto abierto
3. «Send»: todas las respuestas salen juntas
4. «Hide» esconde la banda hasta la siguiente respuesta

### Lo que llega

De cada respuesta sale una línea con el número y el punto entre paréntesis, para que Claude sepa exactamente a qué te refieres:

```
Sobre 1 (Aún faltan 4 imágenes): ya están en la carpeta
Sobre 2 (Esto todavía lo tienes que hacer tú): está hecho
```

### Conviene saber

- Cuenta la última lista con al menos 2 puntos, como máximo 9 botones
- Solo con preguntas: hay un «?» en un punto, en la línea justo antes o en el párrafo justo después (en listas con «-» solo cuenta el párrafo de después). También cuenta un título justo antes como «Lo que necesito de ti:» o «Preguntas abiertas:». Palabras como «Avísame» por sí solas no activan la banda, ni tampoco las instrucciones y listas sin pregunta
- Una pregunta abierta sin lista («¿Cómo lo hago?», «¿Izquierda o derecha?») no recibe banda; la respondes directamente en el campo de entrada.
- Una pregunta de sí/no al final (por ejemplo «¿Te parece bien el plan?») recibe dos botones, **SÍ** y **NO**. Un clic envía «Sí» o «No» de inmediato, también con una lista delante
- Si Claude nombra caminos en líneas propias («Camino 1 (recomendado): …», «Camino 2: …») y pregunta de forma abierta («¿Cómo lo hago?»), hay un botón por camino, **CAMINO 1**, **CAMINO 2**; la recomendación está marcada y un clic envía «Camino 1» de inmediato
- Bajo un informe sin pregunta aparecen los botones **SIGUIENTE TAREA**, **GUARDAR** y **¿QUÉ QUEDA?** tras una tarea terminada, y solo **¿QUÉ QUEDA?** en un estado intermedio; las respuestas cortas no reciben nada. Se detecta por palabras como «verificado», «hecho» o «siguiente». Envían «Muéstrame las tareas que se pueden empezar.», «Guarda ahora: comprobar, commit, pull request.» y «¿Qué queda abierto en esta tarea?». Están pensados para proyectos con lista de tareas y Git
- Los puntos largos se acortan al final de una palabra, la pregunta completa aparece en el campo de respuesta
- Las listas dentro de bloques de código se saltan
- Sin ratón: `Ctrl+X`, luego `Tab` hasta los botones y `Enter`. `Esc` vuelve al campo de escritura
- Los botones no tienen atajos de teclado a propósito: al escribir nunca se activa nada por error
- Cada mensaje nuevo vacía la banda

## El panel /bro

Con `/bro` abres el panel. En la aplicación de escritorio se abre solo con tu primer mensaje.

**Tu ventaja:** También en la aplicación de escritorio, donde no hay línea de estado, ves todo de un vistazo.



```
⚙ trabajando 1m 12s → Bash npm test 8s
contexto ▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱ 58% 116k/200k ▸ terminar la tarea
         · próximo mensaje ≈ 12k tokens
límite   · S | 5h / 23%  · Sem | 7d / 71%
coste    · API | ≈$1.50
activo   read×6  edit×3  bash×2  ◇ pdf  ⌁ github×2  ⬡ Explore
archivos ✎ statusline.py  + hilfe.py  · anleitung.md
```



**Sonidos**
aviso al «esperarte», sonido suave al «terminar»

Sí

Más ▸

Recreado a partir del código de la mod, tal como lo muestra la aplicación de escritorio. Arriba está el estado con el diamante, debajo la caché. En el terminal, el tiempo restante y las tasas de aciertos aparecen como texto en vez de círculos. Una línea por ajuste: a la izquierda el nombre y para qué sirve, a la derecha el interruptor. Los ajustes finos están en «Más».

| | |
|---|---|
| **ESTADO ARRIBA** | Los mismos datos y colores que la línea de estado: diamante, lo que hace Claude, semáforo de contexto, límites y costes, herramientas y archivos usados. Pensado para la aplicación de escritorio; en el terminal es un resumen de la línea de estado. |
| **ESPERÁNDOTE** | Si Claude Code te pide una aprobación o Claude te muestra una selección para hacer clic, aparece enseguida en rojo «esperándote» con lo que hay que aprobar, y en la aplicación de escritorio suena el aviso. En modo automático primero comprueba el revisor automático: mientras comprueba aparece «Auto revisa: <herramienta>» y todo sigue en calma. «esperándote» solo sale cuando de verdad te preguntan a ti. BroStatus reconoce la aprobación mediante los hooks de `settings.json`; si faltan, «esperándote» aparece tras 30 segundos sin inicio. La app de escritorio no devuelve la aprobación: la indicación se mantiene hasta que la herramienta termina. Tras 30 segundos añade por eso «aprobación o en ejecución desde hace …» con un tiempo que sube y, poco antes y después de que caduque la caché, un aviso de caché. |
| **CACHÉ ESTIMADA** | Si la línea de estado está activa, la cuenta atrás de la caché sale exacta de sus datos. En la aplicación de escritorio sin línea de estado, BroStatus calcula 1 hora desde la última petición y añade «estimado». Las tasas de aciertos salen entonces de las respuestas de Claude. |

| BOTÓN / CAMPO | EFECTO |
|---|---|
| ◆▾ / ◆▸ (a la izquierda, antes de los círculos) | Muestra u oculta los interruptores y Más, para que el panel ocupe menos. El estado y los círculos se quedan. Se recuerda. |
| Más ▸ / Más ▾ | Se despliega, en secciones: los tres círculos y los límites, sonidos, idioma, coste y consejo |
| Idioma (dentro de Más) | Deutsch, English, Español, Italiano o Français. Vale para el panel, la banda de respuestas, los avisos y la línea de estado en el terminal; la elección se queda guardada. |
| Moneda (en Más) | Dólar, euro, libra o franco. El valor de API de la línea «coste» se convierte con un tipo incluido (tipo de referencia del BCE, la fecha aparece bajo la selección). Tu propio tipo va en `config.json` ( `currency_rate` : cuántas unidades de la moneda es un dólar). Sigue siendo una estimación (≈), no un importe de factura. Los tipos están en `kurse.json` y se actualizan con las versiones nuevas; BroStatus no los descarga de internet. |
| Cerrar | Otra vez `/bro` o `Esc`, en la aplicación de escritorio la cruz del panel. Todo lo que esté activado sigue funcionando |
| · S \| 5h / … · Sem \| 7d / … (línea «límite») | S es el límite de sesión (las 5 horas en curso), Sem el límite semanal (7 días), cada uno con lo que ya has gastado. Amarillo desde el 65 %, rojo desde el 85 %. Igual en la línea de estado. |
| · API \| $ (línea «coste») | API es el valor de API estimado de la sesión (Claude Code lo calcula localmente a partir de los tokens al precio de lista; con suscripción no lo pagas). El signo ≈ significa: estimación, no un valor exacto. Para leer la caché BroStatus usa el factor del modelo que estás usando (según la lista de precios de Anthropic, a 29/10/2026: 0,05 en Opus 5.5 y Sonnet 5.5, 0,025 en Fable 5.1 y Mythos 5.1, en los demás 0,1). Si BroStatus no conoce el modelo, usa 0,1; en modelos nuevos el factor puede diferir. |
| Sonidos (solo aplicación de escritorio) | Línea propia con piloto (**verde** activado, **rojo** desactivado) y Sí/No, activada por defecto: aviso llamativo en «esperándote», sonido suave en «listo» tras un trabajo largo. En «Más»: uno de tres sonidos por ocasión, «Sonido final desde duración» y «Repetir sonido» (No, cada 2 o cada 5 min): un sonido no oído vuelve como máximo 3 veces, hasta que escribes, apruebas o eliges. Con muestra y botones de prueba. «Volumen del sonido» tiene los niveles 25, 50, 75 y 100 % (por defecto 50 %) y vale para todos los sonidos, también los propios; funciona en Windows, en Mac y Linux la app reproduce el sonido con su propio volumen. En el terminal, la campana del terminal. |
| · próximo mensaje ≈ … tokens (bajo «contexto», solo en el panel) | Lo que cuesta el siguiente mensaje según el estado de la caché: *caliente* más o menos una décima parte (menos en algunos modelos) del contexto, *fría* la reconstrucción. Es una estimación; sin datos de caché no hay cifra. |

### Colores en el panel

- ● **caliente**: todo bien, el círculo muestra el tiempo restante
- ● **caduca pronto**: menos de 10 min (caché de 1 h) o 1 min

- ● **caduca enseguida**: menos de 2 min o 30 s, ● **frío**: caducada
- Anillos: verde desde el 80 %, amarillo desde el 50 %, si no rojo

Claude Code compacta por sí solo tras más o menos 1 hora de inactividad; se desactiva con `"idleCompaction": false` en `settings.json`. Fijar la duración de la caché: `"promptCacheTtl": "1h"`.

## Señales de listo y de espera

Para que no tengas que mirar a cada rato.

### Título de la ventana

El título de la ventana del terminal muestra el estado, aunque esté en segundo plano:

```
◆ mi-proyecto · ⚙ Bash npm test
◆ mi-proyecto · ⏸ esperándote
◆ mi-proyecto · ◔ trabajando en segundo plano (2)
◆ mi-proyecto · ✓ listo (3m 20s)
```

### Sonido y aviso en el terminal

Cuando Claude te espera, o cuando una respuesta ha durado al menos 15 segundos:

```
💎 ✓ Claude ha terminado – mi-proyecto (3m 20s)
Claude te está esperando (mi-proyecto)
```

El límite se ajusta con `notify_min_secs`.

### Telegram con un bot propio

Opcional. Práctico si manejas Claude desde el móvil. El mensaje incluye el nombre del proyecto y el aviso (terminado con duración, o esperándote). BroStatus solo añade el comienzo de la respuesta de Claude (400 caracteres) y el nombre del ordenador si los activas en `config.json` ( `telegram_include_answer`, `telegram_include_host`, desactivados por defecto).

Configurar: véase Ajustes.

### ¿Quieres usar tus propios sonidos?

Funciona en Windows en la aplicación de escritorio. Tus sonidos se quedan contigo, una actualización no los borra.

1. En la carpeta de BroStatus `C:\Users\<Name>\.claude\brostatus` (junto a `state`) crea una carpeta `toene`.
2. Pon dentro tus sonidos como archivos WAV: `warten—<Nombre>.wav` para el aviso, `fertig—<Nombre>.wav` para el sonido final, por ejemplo `fertig-Gong.wav`.
3. Inicia una sesión nueva. En `/bro` → «Más» → Sonidos aparece «Gong» detrás de los sonidos incluidos. La primera vez tu sonido pasa a ser el predeterminado.

Usa solo sonidos que tengas permiso para usar. En Mac y Linux, BroStatus solo reproduce los sonidos incluidos.

## Instalación

Windows | Mac | Linux. Un instalador por sistema. Configura la línea de estado, las señales y la mod, y antes hace una copia de seguridad.



Reconoces los instaladores por este icono. En Windows lo lleva el `.exe`, en el Mac la aplicación. Tras la instalación, también la carpeta `brostatus` tiene este icono.

### Windows



**BroStatus-Install.exe**
Basta con doble clic

1. Descarga `BroStatus-Install.exe`
2. Haz doble clic
3. Si aparece «Windows protegió su PC»: **Más información**, luego **Ejecutar de todas formas**
4. Si falta Python, la ventana lo instala sola (1–2 minutos)
5. Elige la carpeta o pulsa `Enter`
6. Con «✓ Fertig!» (el instalador lo muestra en alemán) pulsa una tecla
7. Reinicia Claude Code, mejor en el **Windows Terminal**

Como alternativa hay `BroStatus-Install.cmd`, que hace lo mismo. Muestra el símbolo normal de CMD.

### Mac



**BroStatus-Install.app**
en `BroStatus-Mac.zip`

1. Descarga `BroStatus-Mac.zip` y descomprímelo con doble clic
2. La primera vez, **clic derecho** en `BroStatus-Install.app` → **Abrir** → **Abrir**
3. ¿No hay botón Abrir? **Ajustes del Sistema** → **Privacidad y seguridad** → **Abrir igualmente**
4. Si falta Python: en el terminal `xcode-select --install`, y otra vez desde el principio
5. Con «✓ Fertig!» pulsa `Enter` y reinicia Claude Code

### Linux

**brostatus-install.sh**
Script para el terminal, sin icono propio

1. Copia `brostatus-install.sh` al ordenador, por ejemplo desde el PC (NAME = tu usuario, SERVER = nombre o dirección IP del servidor):

```
scp brostatus-install.sh NAME@SERVER:~/
```

2. Ejecútalo:

```
bash brostatus-install.sh
```

3. Si falta Python: `sudo apt install -y python3`, y otra vez desde el principio
4. Reinicia Claude Code

La línea de estado solo la ves en el terminal, es decir, por SSH. Con Remote Control desde el móvil recibes en su lugar el aviso de Telegram.

**CARPETA**

Lo normal es `~/.claude/brostatus`, en Windows `C:\Users\<Name>\.claude\brostatus`. Si indicas otra carpeta, todo queda dentro de la subcarpeta `brostatus`, y las actualizaciones se quedan allí. En Mac y Linux: `--ziel <ordner>`.

**AJUSTES**

El `settings.json` de Claude Code normalmente está en `~/.claude` (o en la carpeta que indique `CLAUDE_CONFIG_DIR`). El instalador anota allí la línea de estado, los hooks y la mod, y antes hace una copia de seguridad.

## Comprobar que todo funciona

Tras el reinicio, en tres pasos.

**Primero inicia Claude Code, luego /bro:** `/bro` y `/plugin` son comandos *dentro* de Claude Code, no de PowerShell ni del terminal. Abre el terminal con normalidad (no como administrador), ve a la carpeta del proyecto ( `cd D:\mein-projekt` ) y escribe `claude`. En el primer inicio, Claude Code te pide iniciar sesión una vez en el navegador. En la aplicación de escritorio, basta con empezar una sesión nueva.

**1 · LÍNEA DE ESTADO**

Abajo aparecen el diamante, «○ bereit» y las líneas de información, al principio en alemán. En `/bro` → Más → Idioma cambias a Español y se ve así:



```
○ preparado · opus/high
contexto▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱ 46% 92k/200k
caché   ◷ 54:58 1h   aciertos 99% última · 90% sesión   límite · S | 5h / 23%  · Sem | 7d / 71%   #tools_c…
activo  –
archivos–
```

**2 · MOD CARGADO**

Escribe `/plugin`. Bajo las pestañas pone «1 mod active · bro-status» (con más mods propios, un número mayor).

```
> /plugin
1 mod active · bro-status
```

Extracto, recreado

**3 · BOTONES**

Pídele a Claude que te haga dos preguntas en una lista numerada. Encima del campo de escritura aparece «◆ Respuestas».

```
◆ Respuestas  pulsa un botón  Send  Hide
Elegir color
Logo a la izquierda o derecha
```

Extracto, recreado

## Ajustes

En `~/.claude/brostatus/config.json`. Los cambios se aplican enseguida, sin reiniciar (solo tu propio tipo de cambio se lee al iniciar la mod).

| CLAVE | VALOR NORMAL | SIGNIFICADO |
|---|---|---|
| `notify_min_secs` | 15 | Desde cuántos segundos de trabajo llega un aviso de «listo» |
| `terminal_notify` | true | Sonido y aviso en el terminal |
| `window_title` | true | Estado en el título de la ventana |
| `telegram_bot_token` | vacío | Token de tu bot para los avisos de Telegram |
| `telegram_chat_id` | vacío | Tu ID de chat de Telegram |
| `logo` | true | Mostrar el diamante a la izquierda |
| `colors` | "truecolor" | Ponlo en "256" si los colores se ven mal |
| `ctx_stages` | [50, 70, 85] | Límites del semáforo de contexto en % (amarillo, naranja, rojo) |
| `files` | "compact" | "compact" = una línea, "tree" = árbol de archivos, "off" = apagado |
| `tree_max_lines` | 8 | Como máximo estas líneas para el árbol de archivos |
| `max_items` | 5 | Entradas por tipo en la línea Activo |

### Configurar Telegram con un bot propio

1. En Telegram, crea tu propio bot con `@BotFather` y anota el token
2. Envía cualquier mensaje a tu bot en Telegram
3. Abre esto en el navegador, cambiando `<TOKEN>` por el token del bot: `https://api.telegram.org/bot<TOKEN>/getUpdates`
4. Tu ID de chat está en `"chat":{"id": …}`

5. Anota el token y el ID de chat en `config.json` :

```
"telegram_bot_token": "123456:ABC...",
"telegram_chat_id": "deine-chat-id"
```

## Actualizaciones y solución de problemas

Para actualizar, solo tienes que ejecutar el nuevo archivo instalador. Tu `config.json` se conserva.

| PROBLEMA | SOLUCIÓN |
|---|---|
| `claude` no se reconoce | Claude Code está instalado, pero no está en el PATH. Ejecútalo una vez en PowerShell y luego vuelve a abrir PowerShell: `[Environment]::SetEnvironmentVariable('Path', [Environment]::GetEnvironmentVariable('Path','User') + ";$env:USERPROFILE\.local\bin", 'User')` |
| `/bro` no se reconoce (mensaje rojo en PowerShell) | Todavía estás en PowerShell. Primero inicia `claude` y luego escribe `/bro` en Claude Code |
| En la aplicación de escritorio, `Enter` tras `/bro` elige otra sugerencia (p. ej. un skill) | Tras `/bro`, el menú de sugerencias solo muestra skills, no el comando de la mod. Pulsa primero `Esc` y luego `Enter`. En una sesión recién iniciada, la mod arranca con tu primer mensaje |
| «Browser didn't open? ... Paste code here» | Inicio de sesión de Claude Code, solo la primera vez: abre el enlace, inicia sesión con tu cuenta de Claude, pega el código que aparece en el terminal (clic derecho) y pulsa `Enter`. No compartas el código en ningún otro sitio |
| Claude Code arranca en `C:\WINDOWS\system32` | No abras PowerShell como administrador y antes ve con `cd` a la carpeta de tu proyecto |
| Cuadritos en vez del diamante | Usa el Windows Terminal en vez de la antigua ventana cmd, o pon `"logo": false` |
| Los colores se ven raros | Pon `"colors": "256"` en `config.json` |
| No hay línea de estado | Reinicia Claude Code y confía en la carpeta (confirma la pregunta de confianza) |
| `/plugin` no muestra bro-status | Comprueba la versión con `claude --version`, la probada es 2.1.310 (versiones anteriores sin comprobar). Si no, `claude update` |
| Llegar a los botones sin ratón | `Ctrl+X`, luego `Tab` y `Enter` |
| Falta Python en Windows | Instálalo desde python.org, marca «Add python.exe to PATH» y ejecuta el instalador otra vez |
| Falta Python en Linux | `sudo apt install -y python3` |
| «Da ist etwas schiefgelaufen» (mensaje del instalador, en alemán: algo ha salido mal) | Pega una captura de pantalla en tu sesión de Claude Code y pide ayuda, o infórmalo como issue |

### Probar la línea de estado por separado

Si aparece el diamante con «○ preparado» (o «○ bereit»), BroStatus probablemente está bien y el problema está más bien en Claude Code. El comando vale para Mac y Linux; en Windows usa PowerShell con `python` en lugar de `python3`.

```
echo '{"session_id":"test","model":{"display_name":"Opus"}}' |
python3 ~/.claude/brostatus/statusline.py
```

### Desinstalar

Mac y Linux:

```
python3 ~/.claude/brostatus/install.py --uninstall
```

Windows (PowerShell):

```
python $HOME\.claude\brostatus\install.py --uninstall
```

Quita todo de `settings.json`, con copia de seguridad. La carpeta `brostatus` se queda, con tu `config.json` (ahí está tu token de Telegram, si lo configuraste). Bórrala a mano cuando ya no la necesites.

BroStatus es un proyecto de DiamondSash (github.com/diamondsash/bro-status). Creado para Claude Code, sin servidores propios y no gasta tokens para el indicador. Licencia: MIT con Commons Clause (uso gratuito, prohibida la venta). Sin vínculo con Anthropic. Claude y Claude Code son marcas de Anthropic.